

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №1
з курсу «Хмарні технології та великі дані»
на тему:
«Розмикач ланцюжка»

Виконав:
студент групи Фел-32
Трофимов Володимир
Перевірив:
доц. Любунь З.М.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи	4
Теоретичні відомості	5
1. Графи та їх основні поняття	Помилка! Закладку не визначено.
2. Основні види графів	Помилка! Закладку не визначено.
2.1. Неорієнтований граф	Помилка! Закладку не визначено.
2.2. Орієнтований граф	Помилка! Закладку не визначено.
3. Дерево.....	Помилка! Закладку не визначено.
4. Пошук у графах	Помилка! Закладку не визначено.
5. Пошук у ширину	Помилка! Закладку не визначено.
6. Черга у BFS.....	Помилка! Закладку не визначено.
7. Алгоритм BFS	Помилка! Закладку не визначено.
8. Відновлення знайденого шляху	Помилка! Закладку не визначено.
9. BFS та найкоротший шлях	Помилка! Закладку не визначено.
Інтерфейс та принцип роботи програми	Помилка! Закладку не визначено.
Результати дослідження роботи програми.....	7
Дослідження 1: зміна напрямку обходу вершин на неорієнтованому графі.	Помилка! Закладку не визначено.
Дослідження 2: зміна початку та кінця пошуку місцями.	Помилка! Закладку не визначено.
Дослідження 3: Видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф.	Помилка! Закладку не визначено.
Дослід 4: Використання пошуку вшир на різних видах графів.	Помилка! Закладку не визначено.
Висновок	12
Код програми	13

Тема

Запобігання каскадним відмовам при збоях віддалених сервісів, швидко «відсікаючи» несправний шлях виклику.

Мета

Дослідження принципу роботи патерну Circuit Breaker та його реалізація мовою TypeScript. Необхідно реалізувати скінченний автомат із трьома станами CLOSED, OPEN та HALF_OPEN, забезпечити обробку помилок і тайм-аутів, а також перевірити коректність роботи за допомогою автоматизованих тестів.

Вхідні дані

Параметр	Значення	Призначення
failureThreshold	3	кількість послідовних помилок, після яких Circuit Breaker переходить у OPEN
halfOpenMaxCalls	2	максимальна кількість пробних викликів у стані HALF_OPEN
openStateDuration	5000 мс	тривалість перебування у стані OPEN перед переходом до HALF_OPEN
timeoutPerCall	1000 мс	максимальний час виконання одного виклику

Завдання

Розробити програмну реалізацію патерну Circuit Breaker мовою TypeScript для захисту системи від повторних викликів несправного або недоступного зовнішнього сервісу.

Реалізація повинна функціонувати як скінченний автомат із трьома станами:

- CLOSED — нормальний режим роботи, у якому виклики дозволені;
- OPEN — стан блокування викликів після досягнення встановленого порогу помилок;
- HALF_OPEN — проміжний стан, у якому після завершення періоду блокування виконуються обмежені пробні виклики для перевірки відновлення сервісу.

Необхідно реалізувати:

- інтерфейс ICircuitBreaker з методами отримання поточного стану та виконання операції;
- конфігураційні параметри failureThreshold, halfOpenMaxCalls, openStateDuration та timeoutPerCall;
- накопичення послідовних помилок у стані CLOSED;

- автоматичний перехід із CLOSED у OPEN після досягнення порогу помилок;
- блокування нових викликів у стані OPEN та генерацію відповідної помилки;
- перехід із OPEN у HALF_OPEN після завершення встановленого періоду очікування;
- обмеження кількості пробних викликів у стані HALF_OPEN;
- повернення до CLOSED у разі успішного проходження пробних викликів;
- повторний перехід до OPEN у разі помилки або тайм-ауту під час перевірки;
- обмеження часу виконання кожного виклику за допомогою тайм-ауту;
- коректне очищення таймерів після завершення операції;
- перевірку коректності вхідних параметрів конфігурації;
- створення демонстраційного HTTP-сервера, за допомогою якого можна перевірити роботу механізму за допомогою HTTP-запитів.

Критерії виконання роботи

Лабораторна робота вважається виконаною за умови реалізації та перевірки основної функціональності патерну Circuit Breaker.

1. Реалізація скінченного автомата

Програмна реалізація повинна містити три стани: CLOSED, OPEN та HALF_OPEN. Повинні бути реалізовані коректні переходи між станами залежно від результату виконання операцій.

2. Робота стану CLOSED

У стані CLOSED виклики зовнішньої операції повинні дозволятися. Успішний виклик повинен скидати лічильник послідовних помилок. Кожна помилка повинна збільшувати лічильник. Після досягнення значення `failureThreshold` Circuit Breaker повинен перейти у стан OPEN.

3. Робота стану OPEN

У стані OPEN нові виклики зовнішньої операції не повинні виконуватися. Замість цього повинен генеруватися `CircuitBreakerOpenError`. Після завершення `openStateDuration` повинна бути забезпечена можливість переходу до стану HALF_OPEN.

4. Робота стану HALF_OPEN

У стані HALF_OPEN повинна виконуватися обмежена кількість пробних викликів. Успішне виконання необхідних пробних викликів повинно переводити Circuit Breaker у стан CLOSED. Помилка або тайм-аут пробного виклику повинні переводити Circuit Breaker назад у OPEN.

5. Обмеження пробних викликів

Повинно бути забезпечено обмеження одночасних пробних викликів у стані HALF_OPEN. При перевищенні дозволеної кількості викликів повинна генеруватися `CircuitBreakerHalfOpenExhaustedError`.

6. Обробка тайм-аутів

Кожна операція повинна мати максимальний час виконання, визначений параметром `timeoutPerCall`. Якщо операція не завершилася протягом цього часу, повинен генеруватися `TimeoutError`, а така ситуація повинна враховуватися як невдалий виклик.

7. Перевірка конфігурації

При створенні `Circuit Breaker` повинні перевірятися значення конфігураційних параметрів. Некоректні значення повинні призводити до генерації відповідної помилки.

8. Демонстрація роботи

Коректність роботи реалізації повинна бути продемонстрована за допомогою HTTP-сервера та відповідних HTTP-запитів, які показують зміни станів `Circuit Breaker` та реакцію системи на успішні виклики, помилки й тайм-аути.

Теоретичні відомості

`Circuit Breaker` — це програмний патерн, який використовується для підвищення відмовостійкості розподілених систем. Його основне призначення полягає у припиненні повторних викликів сервісу, який тимчасово працює некоректно або є недоступним.

Проблема, яку вирішує `Circuit Breaker`, виникає у системах, де один сервіс залежить від іншого. Якщо зовнішній сервіс починає повертати помилки або не відповідає, велика кількість повторних запитів може призвести до накопичення незавершених операцій, збільшення часу очікування та додаткового навантаження на систему. У результаті одна несправність може поширитися на інші компоненти системи, що називається каскадною відмовою.

`Circuit Breaker` запобігає цьому шляхом переходу між трьома основними станами: CLOSED, OPEN та HALF_OPEN.

Стан CLOSED

CLOSED є нормальним робочим станом `Circuit Breaker`. Усі виклики зовнішньої операції дозволені.

При успішному виконанні операції лічильник послідовних помилок скидається. Якщо операція завершується помилкою, лічильник збільшується. Коли кількість послідовних помилок досягає значення `failureThreshold`, `Circuit Breaker` переходить у стан OPEN.

Таким чином, параметр `failureThreshold` визначає, скільки послідовних невдалих викликів система допускає перед блокуванням подальших запитів.

Стан OPEN

У стані OPEN виклики зовнішньої операції блокуються. Функція зовнішнього сервісу фактично не викликається, а клієнту одразу повертається помилка `Circuit Breaker`.

Це дозволяє не витрачати ресурси на виклики сервісу, який уже продемонстрував свою несправність.

`Circuit Breaker` залишається у стані OPEN протягом часу, визначеного параметром `openStateDuration`. Після завершення цього періоду система повинна перевірити, чи відновився зовнішній сервіс.

У реалізації використовується лінійний перехід: окремий фоновий таймер для зміни стану не запускається. Перехід до HALF_OPEN перевіряється під час наступного звернення до `Circuit Breaker`.

Стан HALF_OPEN

HALF_OPEN використовується для перевірки відновлення зовнішнього сервісу.

Після завершення періоду блокування `Circuit Breaker` дозволяє виконати обмежену кількість пробних операцій. Параметр `halfOpenMaxCalls` визначає допустиму кількість таких викликів.

Якщо пробні виклики завершуються успішно, `Circuit Breaker` робить висновок, що зовнішній сервіс відновив роботу, і переходить у CLOSED.

Якщо хоча б один пробний виклик завершується помилкою або тайм-аутом, `Circuit Breaker` знову переходить у OPEN, оскільки сервіс усе ще вважається несправним.

Тайм-аут операції

Окрім явних помилок, зовнішній сервіс може просто не відповідати. Для запобігання нескінченному очікуванню використовується механізм тайм-ауту.

Параметр `timeoutPerCall` визначає максимальний час виконання однієї операції. Якщо операція не завершилася за цей час, генерується `TimeoutError`.

У TypeScript тайм-аут можна реалізувати за допомогою `Promise.race()`, де одночасно очікується результат основної операції та завершення таймера.

Наприклад, концептуально:

```
Promise.race([operation, timeout])
```

Першою завершується або основна операція, або таймер. Після завершення виклику таймер необхідно очистити, щоб уникнути зайвих активних таймерів.

Основні помилки Circuit Breaker

У реалізації використовуються спеціальні типи помилок:

- `CircuitBreakerOpenError` — виклик заблокований, оскільки `Circuit Breaker` перебуває у стані `OPEN`;
- `CircuitBreakerHalfOpenExhaustedError` — досягнуто допустимої кількості пробних викликів у `HALF_OPEN`;
- `TimeoutError` — операція не завершилася протягом встановленого часу.

Розділення помилок дозволяє HTTP-серверу по-різному обробляти різні ситуації. Наприклад, заблокований `Circuit Breaker` може повертати HTTP 503, перевищення кількості пробних викликів — HTTP 429, а тайм-аут — HTTP 504.

Скінчений автомат Circuit Breaker

Основні переходи між станами можна представити так:

CLOSED → OPEN

Перехід відбувається після досягнення `failureThreshold` послідовних помилок.

OPEN → HALF_OPEN

Перехід відбувається після завершення `openStateDuration`.

HALF_OPEN → CLOSED

Перехід відбувається після успішного проходження пробних викликів.

HALF_OPEN → OPEN

Перехід відбувається при помилці або тайм-ауті пробного виклику.

Отже, `Circuit Breaker` працює як контрольований механізм ізоляції несправного сервісу. Він дозволяє швидко відмовляти у виконанні запитів під час аварійного стану, періодично перевіряти можливість відновлення та автоматично повертатися до нормальної роботи після відновлення зовнішньої залежності.

Структура програмної реалізації

Програмна реалізація лабораторної роботи виконана мовою **TypeScript** із використанням середовища виконання **Node.js**. Програма складається з основного модуля `Circuit Breaker` та HTTP-сервера, який використовується для демонстрації його роботи.

Структура проєкту має такий вигляд:

Lab_1/

```
|— src/
|   |— CircuitBreaker.ts
|   └— server.ts
|— package.json
|— tsconfig.json
└— vitest.config.ts
```

Основна логіка патерну Circuit Breaker реалізована у файлі `src/CircuitBreaker.ts`. У ньому визначено три можливі стани автомата: `CLOSED`, `OPEN` та `HALF_OPEN`. Також у цьому модулі описані параметри конфігурації, інтерфейс `ICircuitBreaker` та спеціальні класи помилок.

До параметрів конфігурації належать:

- `failureThreshold` — кількість послідовних невдалих викликів, після якої Circuit Breaker переходить у стан `OPEN`;
- `halfOpenMaxCalls` — максимальна кількість пробних викликів у стані `HALF_OPEN`;
- `openStateDuration` — тривалість перебування у стані `OPEN` у мілісекундах;
- `timeoutPerCall` — максимальний час виконання одного виклику.

Метод `call<T>()` є основним методом класу `CircuitBreaker`. Перед виконанням операції він перевіряє поточний стан автомата. Якщо стан `OPEN`, виклик блокується та генерується `CircuitBreakerOpenError`. У стані `HALF_OPEN` додатково контролюється кількість дозволених пробних викликів.

Для реалізації обмеження часу виконання операції використовується конструкція `Promise.race()`. Вона порівнює завершення основної операції з таймером. Якщо операція не завершується протягом заданого часу, генерується `TimeoutError`, який розглядається як невдалий виклик та впливає на стан `Circuit Breaker`.

Перехід між станами реалізовано без використання фонових таймерів. Після переходу в `OPEN` зберігається час, після якого дозволено наступну спробу. Під час наступного звернення до `Circuit Breaker` перевіряється поточний час і, якщо період блокування завершився, виконується перехід до `HALF_OPEN`.

Файл `src/server.ts` реалізує HTTP-сервер для демонстрації роботи `Circuit Breaker`. Сервер використовує клас `CircuitBreaker` та надає декілька HTTP-ендпоінтів:

- GET /state — повертає поточний стан Circuit Breaker;
- GET /health — використовується для перевірки стану сервісу;
- POST /call — виконує операцію через Circuit Breaker;
- POST /restart — створює новий екземпляр Circuit Breaker та повертає його до початкового стану.

Ендпоінт /call підтримує декілька режимів роботи віддаленої операції. У режимі success операція завершується успішно, у режимі failure генерується помилка, а в режимі timeout створюється операція, яка не завершується протягом встановленого часу. Це дозволяє продемонструвати реакцію Circuit Breaker на різні типи результатів виконання.

HTTP-сервер також виконує перетворення внутрішніх помилок Circuit Breaker у відповідні HTTP-відповіді. Зокрема, при відкритому Circuit Breaker клієнт отримує статус 503 Service Unavailable, при перевищенні кількості пробних викликів у стані HALF_OPEN — 429 Too Many Requests, а при перевищенні часу виконання операції — 504 Gateway Timeout.

Тестування програмної реалізації

Перший етапом буде запуск сервера та перевірка його стану.

```
$ curl -s http://localhost:3000/state
{"ok":true,"state":"CLOSED","restartCount":0}
```

Рис.1 – Відповідь сервера при перевірці стану

Виконаємо до нього запит.

```
$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"hello"}'
{"ok":true,"state":"CLOSED","result":"hello","restartCount":0}
```

Рис.2 – Відповідь сервера при виконанні успішного запиту

Тепер переводимо Circuit Breaker в стан OPEN. Відразу після цього виконаємо запит, який мав би бути успішним.

```

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies (main)
$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"failure","value":"service unavailable"}'
{"ok":false,"state":"CLOSED","error":"service unavailable","errorName":"Error","restartCount":0}
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies (main)
$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"failure","value":"service unavailable"}'
{"ok":false,"state":"CLOSED","error":"service unavailable","errorName":"Error","restartCount":0}
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies (main)
$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"failure","value":"service unavailable"}'
{"ok":false,"state":"OPEN","error":"service unavailable","errorName":"Error","restartCount":0}
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies (main)
$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"hello"}'
{"ok":false,"state":"OPEN","error":"Circuit breaker is open","errorName":"CircuitBreakerOpenError","restartCount":0}

```

Рис.3 – Відповідь сервера при переведенні Circuit Breaker в стан OPEN

З рисунка видно, що Circuit Breaker в стані OPEN. Почекаємо 5 секунд і побачимо зміну стану на HALF_OPEN. Тепер виконаємо 3 запити, які відбуватимуться водночас.

```

$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"timeout"}' & curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"timeout"}' & sleep 0.1; curl -i -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"probe-3"}'; wait
[1] 5676
[2] 5677
HTTP/1.1 429 Too Many Requests
content-type: application/json
Date: Tue, 08 Sep 2026 19:04:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"ok":false,"state":"HALF_OPEN","error":"Circuit breaker has no available half-open probe capacity","errorName":"CircuitBreakerHalfOpenExhaustedError","restartCount":0}{"ok":false,"state":"OPEN","error":"Circuit breaker call timed out","errorName":"TimeoutError","restartCount":0}[1]- Done
  curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"timeout"}'
{"ok":false,"state":"OPEN","error":"Circuit breaker call timed out","errorName":"TimeoutError","restartCount":0}[2]+ Done
  curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"timeout"}'

```

Рис.4 – Відповідь сервера при виконанні трьох паралельних запитів при обмеженні в два запити за раз для стану HALF_OPEN

Як і очікувалося – помилка 429. Тепер виконаємо два успішні запити і стан Circuit Breaker має стати CLOSE.

```
$ curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"probe-1"}' & curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"probe-2"}' & wait
[1] 5690
[2] 5691
{"ok":true,"state":"HALF_OPEN","result":"probe-1","restartCount":0}[1]- Done
curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"probe-1"}'
{"ok":true,"state":"CLOSED","result":"probe-2","restartCount":0}[2]+ Done
curl -s -X POST http://localhost:3000/call -H 'content-type: application/json' -d '{"mode":"success","value":"probe-2"}'

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies (main)
$ curl -s http://localhost:3000/state
{"ok":true,"state":"CLOSED","restartCount":0}
```

Рис.5 – Відповідь сервера при виконанні двох успішних запитів

Висновок

У ході виконання лабораторної роботи було досліджено принцип роботи патерну Circuit Breaker та реалізовано його мовою TypeScript із використанням середовища виконання Node.js. Основною метою було створення механізму захисту системи від каскадних відмов під час взаємодії з несправними або недоступними віддаленими сервісами.

У програмній реалізації було створено скінченний автомат із трьома станами: CLOSED, OPEN та HALF_OPEN. Було реалізовано накопичення послідовних помилок і автоматичний перехід до стану OPEN після досягнення заданого порогу. У стані OPEN нові виклики блокуються, що дозволяє уникнути додаткового навантаження на несправний сервіс. Після завершення заданого періоду очікування Circuit Breaker переходить у стан HALF_OPEN, де виконується обмежена кількість пробних викликів для перевірки можливості відновлення сервісу.

Також було реалізовано обмеження кількості одночасних пробних викликів, обробку помилок і тайм-аутів. Для обмеження часу виконання операцій використовується механізм Promise.race(), а після завершення операції відповідний таймер очищується. Окремі типи помилок дозволяють розрізняти блокування викликів, перевищення кількості пробних операцій та перевищення часу очікування.

Для практичної демонстрації роботи механізму було створено HTTP-сервер, який дозволяє перевіряти поточний стан Circuit Breaker, виконувати успішні та невдалі операції, моделювати тайм-аут і перезапускати механізм. Під час тестування за допомогою HTTP-запитів було продемонстровано перехід CLOSED → OPEN після накопичення помилок, блокування запитів у стані OPEN, перехід OPEN → HALF_OPEN після завершення періоду очікування, обмеження пробних викликів та повернення до CLOSED після успішного відновлення сервісу.

Таким чином, поставлену мету лабораторної роботи було досягнуто. Реалізований Circuit Breaker забезпечує контрольований захист від повторних викликів несправного сервісу, дозволяє швидко відмовляти у виконанні запитів під час аварійного стану та автоматично перевіряти можливість відновлення зовнішньої залежності.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_1 та має повну інструкцію запуску даної програми.